

你现在是该项目的首席软件架构师、机器学习工程师、网络安全工程师、测试工程师和技术文档工程师。

你只知道以下项目题目，不应受任何既有 Proposal、旧代码或预设技术方案限制：

Design and Implementation of an Autonomous Threat Hunting System Using Machine Learning

请从零开始完成需求分析、架构设计、开发、测试、实验评估、前端演示、部署和文档。

不要把该项目简单实现成“训练一个分类模型并显示预测结果”。项目必须体现完整的 threat hunting lifecycle：

1. Security telemetry collection
2. Behavioral feature extraction
3. Known-threat detection
4. Unknown/anomalous behavior detection
5. Alert correlation
6. Threat hypothesis generation
7. Investigation and case management
8. Analyst feedback
9. Model evaluation and controlled retraining
10. Interactive frontend demonstration

系统最终应形成一个可运行、可测试、可复现、可展示的硕士毕业设计原型。

1. Project Definition

项目暂定名称：

AegisHunt

项目定位：

AegisHunt 是一个使用机器学习进行自主威胁狩猎的安全分析平台。系统从网络流量或结构化安全遥测中提取行为特征，通过监督式分类模型检测已知恶意行为，通过无监督异常检测模型发现偏离正常基线的未知活动，再对相关告警进行关联、评分和调查假设生成。

系统不应自动声称已经确认攻击，而应输出：

- suspicious behavior
- supporting evidence
- confidence score
- associated entities
- possible attack category
- investigation hypothesis
- recommended investigation steps

系统应提供前端界面，让用户能够：

- 查看系统状态；
 - 导入或回放 PCAP；
 - 查看流量和告警；
 - 查看异常解释；
 - 查看 threat hunting hypotheses；
 - 创建和管理 investigation cases；
 - 标记 true positive / false positive；
 - 查看模型和实验结果；
 - 重新训练模型。
-

2. Core Research Concept

系统采用双检测引擎：

Engine A: Supervised Detection

用于识别训练数据中已有的攻击类别。

候选模型：

- Random Forest
- HistGradientBoostingClassifier
- Logistic Regression baseline
- DummyClassifier baseline

主模型由实验结果决定，而不是提前假设 Random Forest 必须最好。

模型选择应综合：

- F1-score
- Recall
- Precision
- PR-AUC
- False Positive Rate
- Inference latency
- Model size
- Explainability
- Stability across data splits

Engine B: Unsupervised Anomaly Detection

用于识别偏离正常行为基线的活动。

主模型：

- Isolation Forest

可选比较模型：

- Local Outlier Factor，离线评估；
- One-Class SVM，小规模实验；
- Autoencoder，仅在时间和资源允许时作为扩展。

无监督模型优先使用 benign-only training strategy，即仅使用正常数据学习行为基线。

Fusion Engine

监督模型和异常模型输出需要融合为统一风险分数。

建议初始公式：

$\text{combined_risk} = \text{supervised_weight} \times \text{supervised_probability}$

- $\text{anomaly_weight} \times \text{normalized_anomaly_score}$
- $\text{correlation_weight} \times \text{correlation_score}$
- $\text{context_weight} \times \text{contextual_risk}$

所有权重必须放在配置文件中。

不得将风险分数直接解释为攻击发生概率。

3. Threat Hunting Lifecycle

系统必须实现以下流程：

Step 1: Observe

接收以下一种或多种数据：

- PCAP files
- Flow CSV files
- Structured JSON security events
- Optional live network interface capture

PCAP replay 是必须实现的稳定演示方式。

Live capture 是增强功能，不是系统正常运行的前置条件。

系统不需要默认扫描任何 target，也不需要固定使用 127.0.0.1。

Step 2: Detect

从原始数据提取行为特征并运行：

- supervised classifier;

- anomaly detector;
- optional deterministic behavioral rules。

Step 3: Correlate

将一定时间窗口内涉及相同实体的告警关联起来。

实体可以包括：

- source IP
- destination IP
- source host
- destination host
- user
- protocol
- service
- flow/session ID

关联结果形成 correlated alert group。

Step 4: Hypothesize

系统根据证据自动生成结构化 threat hypothesis，例如：

- Suspected reconnaissance activity
- Possible credential abuse
- Possible brute-force pattern
- Possible beaconing behavior
- Possible command-and-control pattern
- Possible data exfiltration pattern
- Possible denial-of-service behavior
- Unclassified behavioral anomaly

Threat hypothesis 必须包含：

- hypothesis_id
- title
- description
- confidence
- severity
- involved_entities
- supporting_alerts
- supporting_features
- time_window
- possible_attack_category
- possible_MITRE_ATT&CK_mapping
- assumptions
- alternative_explanations
- recommended_queries
- recommended_investigation_steps
- status

Threat hypothesis 应由确定性模板和证据生成，不得依赖外部 LLM 才能运行。

可以将 LLM summary 设计成可选扩展，但核心系统必须在没有 API Key 的情况下完整运行。

Step 5: Investigate

用户可从 hypothesis 创建 investigation case。

Case 包含：

- case_id
- title
- priority
- status
- assigned_to
- related_hypotheses
- related_alerts
- evidence
- notes
- analyst_verdict
- created_at
- updated_at
- closed_at

Step 6: Learn

用户可以将告警或案例标记为：

- true_positive
- false_positive
- benign_expected
- needs_more_information

这些反馈写入数据库，并可导出为 future retraining dataset。

不得默认自动在线学习或自动替换生产模型。

重新训练必须由用户显式触发，并保存新模型版本。

4. System Architecture

采用模块化单体架构，避免为了毕设过度引入微服务。

建议架构：

Frontend:

- Streamlit

Backend API:

- FastAPI

Background processing:

- Python worker
- asyncio queue 或轻量任务队列
- 不强制使用 Celery、Redis 或 Kafka

Storage:

- SQLite 作为默认数据库
- SQLAlchemy ORM
- WAL mode
- 设计时保留迁移至 PostgreSQL 的可能性

Machine Learning:

- scikit-learn
- pandas
- numpy
- joblib
- optional SHAP

Traffic processing:

- Scapy
- optional PyShark adapter
- 自定义 bidirectional flow aggregation

Visualization:

- Plotly
- Matplotlib 用于实验报告

Validation:

- Pydantic

CLI:

- Typer

Testing:

- pytest
- pytest-cov

Code quality:

- ruff
- mypy

- pre-commit

Deployment:

- Docker
- Docker Compose
- local Python mode

5. Recommended Repository Structure

aegishunt/ | README.md | LICENSE | pyproject.toml | requirements.txt |
Makefile | Dockerfile | docker-compose.yml | .env.example | .gitignore |
configs/ | application.yaml | detection.yaml | correlation.yaml |
hypothesis.yaml | model_training.yaml | logging.yaml | src/ | aegishunt/
| init.py | cli.py | config.py | schemas.py | constants.py |
logging_config.py | ingestion/ | base.py | pcap_ingestor.py |
csv_ingestor.py | json_ingestor.py | live_ingestor.py | replay_engine.py
| flows/ | packet_parser.py | flow_key.py | flow_state.py |
flow_table.py | flow_finalizer.py | features/ | base.py |
network_features.py | temporal_features.py | directional_features.py |
tcp_features.py | feature_registry.py | feature_validator.py | datasets/ |
registry.py | builder.py | validator.py | splitting.py |
leakage_detection.py | manifests.py | ml/ | supervised/ |
baselines.py | train.py | tune.py | predict.py |
anomaly/ | isolation_forest.py | normalization.py | predict.py |
fusion/ | score_fusion.py | thresholding.py | evaluation/
metrics.py | confidence_intervals.py | robustness.py |
plots.py | explainability/ | global_explanations.py |
local_explanations.py | reason_codes.py | model_registry.py |
detection/ | detection_engine.py | behavioral_rules.py | alert_factory.py
severity.py | correlation/ | entity_index.py |
temporal_correlator.py | correlation_rules.py | alert_groups.py | hunting/
hypothesis_engine.py | hypothesis_templates.py |
evidence_builder.py | mitre_mapping.py | investigation_queries.py |
cases/ | case_service.py | feedback_service.py | export_service.py |
runtime/ | pipeline.py | worker.py | health.py |
telemetry.py | storage/ | database.py | orm_models.py |
migrations.py | repositories/ | api/ | app.py | dependencies.py
routes/ | system.py | ingestion.py | flows.py |
alerts.py | hypotheses.py | cases.py | models.py |
evaluation.py | frontend/ | app.py | api_client.py | components/ |
pages/ | overview.py | data_ingestion.py | traffic_explorer.py |
alerts.py | threat_hunts.py | cases.py | model_lab.py | evaluation.py |
system_health.py | scripts/ | setup.sh | download_dataset.py |
build_dataset.py | train_models.py | run_evaluation.py | seed_demo_data.py |
run_demo.sh | data/ | raw/ | interim/ | processed/ |
sample/ | manifests/ | artifacts/ | models/ | evaluations/ | figures/
logs/ | tests/ | fixtures/ | unit/ | integration/ | e2e/ |
notebooks/ | 01_exploratory_analysis.ipynb | docs/ | system_requirements.md |

|—— architecture.md | |—— threat_model.md | |—— data_model.md | |——
feature_dictionary.md | |—— model_card.md | |—— experiment_protocol.md | |——
deployment.md | |—— demo_guide.md | |—— codex_progress.md |—— reports/ |——
experiment_summary.md |—— limitations.md |—— figures/ |—— tables/

6. Core Data Model

至少定义以下实体：

TelemetrySource

- source_id
- source_type
- filename_or_interface
- ingestion_mode
- status
- started_at
- completed_at
- records_processed
- checksum
- metadata

NetworkFlow

- flow_id
- source_id
- capture_session_id
- first_seen
- last_seen
- duration
- source_ip
- destination_ip
- source_port
- destination_port
- protocol
- forward_packet_count
- backward_packet_count
- forward_bytes
- backward_bytes
- behavioral_features
- ground_truth_label, 可为空
- attack_family, 可为空

DetectionResult

- detection_id
- flow_id
- supervised_label

- supervised_probability
- anomaly_score
- normalized_anomaly_score
- behavioral_rule_score
- combined_risk_score
- severity
- model_versions
- explanation
- detected_at

SecurityAlert

- alert_id
- detection_id
- alert_type
- severity
- title
- description
- involved_entities
- evidence
- reason_codes
- status
- created_at

AlertGroup

- group_id
- alert_ids
- entity_keys
- correlation_score
- first_seen
- last_seen
- summary

ThreatHypothesis

按照前文定义完整实现。

InvestigationCase

按照前文定义完整实现。

AnalystFeedback

- feedback_id
- object_type
- object_id
- verdict
- confidence
- notes

- created_at

ModelVersion

- model_id
 - model_type
 - version
 - algorithm
 - feature_schema
 - training_dataset
 - training_config
 - metrics
 - artifact_path
 - created_at
 - status
-

7. Feature Engineering

系统主要使用不依赖 Payload 的 flow-level behavioral features。

至少实现：

Volume Features

- total_packets
- total_bytes
- forward_packets
- backward_packets
- forward_bytes
- backward_bytes
- packets_per_second
- bytes_per_second
- forward_backward_packet_ratio
- forward_backward_byte_ratio

Packet Size Features

- mean_packet_size
- std_packet_size
- min_packet_size
- max_packet_size
- median_packet_size
- packet_size_q25
- packet_size_q75
- forward_mean_packet_size
- backward_mean_packet_size

Timing Features

- flow_duration
- mean_inter_arrival_time
- std_inter_arrival_time
- min_inter_arrival_time
- max_inter_arrival_time
- median_inter_arrival_time
- iat_q25
- iat_q75
- forward_mean_iat
- backward_mean_iat

TCP Features

- syn_count
- ack_count
- fin_count
- rst_count
- psh_count
- urg_count
- syn_ratio
- rst_ratio
- ack_ratio
- completed_handshake_indicator

Behavioral Features

- connection_burst_score
- periodicity_score
- asymmetry_score
- short_connection_ratio
- failed_connection_indicator
- repeated_destination_indicator
- repeated_port_indicator

IP 地址和原始端口可用于调查和关联，但默认不作为监督模型训练特征。

所有特征必须：

- deterministic;
 - documented;
 - versioned;
 - 无 NaN;
 - 无 Infinity;
 - 训练和实时推理顺序一致;
 - 有 feature_schema.json;
 - 有单元测试。
-

8. Dataset Strategy

系统应支持两类数据：

A. Public Benchmark Dataset

Codex 应选择一个适合网络流量分类的公开数据集作为主要 benchmark。

选择标准：

- 可公开获取；
- 含 benign 和多个 attack families；
- 有明确标签；
- 数据规模适合个人计算机；
- 可转换为统一 feature schema；
- 许可证允许学术使用。

Codex 必须在 docs/dataset_selection.md 中记录：

- 候选数据集；
- 选择理由；
- 下载地址；
- 许可证；
- 原始字段；
- 字段映射；
- 已知限制。

如果数据集无法自动下载：

- 提供下载脚本；
- 提供清晰错误；
- 使用 sample dataset 保证测试和 Demo 可运行；
- 不得伪造完整数据集。

B. Controlled Demo Dataset

创建可复现的小型安全演示数据：

- benign browsing-like flows；
- DNS-like flows；
- file-transfer-like flows；
- scan-like flows；
- brute-force-like repeated connection patterns；
- beacon-like periodic communication；
- exfiltration-like asymmetric transfer；
- connection-flood-like受限行为。

演示数据不得进行真实攻击或漏洞利用。

9. Data Leakage Prevention

严禁简单随机拆分高度相关的流记录。

优先采用：

- capture_session group split;
- scenario group split;
- source file group split;
- time-based split。

必须检查：

- train/validation/test group overlap;
- duplicate rows;
- near duplicates;
- label leakage;
- source file leakage;
- attack family leakage;
- timestamp leakage;
- features directly encoding label。

必须输出：

- dataset_manifest.json
- split_manifest.json
- leakage_report.json
- class_distribution.csv
- feature_statistics.csv

测试集只能在模型和阈值冻结后使用一次进行最终评估。

10. Unknown-Attack Evaluation

项目不得声称保证检测 zero-day attacks。

应使用以下实验模拟“未见攻击行为”：

Leave-One-Attack-Family-Out

每次将一个 attack family 完全排除在监督模型训练之外。

测试：

- supervised model 是否漏检;
- anomaly model 是否能够识别;
- fusion engine 是否改善召回率。

Temporal Holdout

使用较早时间训练，较晚时间测试。

Parameter Shift

训练和测试使用不同的：

- flow duration;
- packet rate;
- packet size pattern;
- connection frequency。

输出：

- known_attack_metrics.csv
 - unseen_attack_metrics.csv
 - leave_one_family_out.csv
 - anomaly_detection_metrics.csv
 - fusion_comparison.csv
-

11. Model Evaluation

至少评估：

Classification Metrics

- Accuracy
- Precision
- Recall
- F1-score
- Macro F1
- Weighted F1
- Balanced Accuracy
- MCC
- ROC-AUC
- PR-AUC
- Specificity
- False Positive Rate
- False Negative Rate
- Confusion Matrix

Anomaly Metrics

- anomaly precision
- anomaly recall
- anomaly F1
- PR-AUC

- score distribution
- threshold sensitivity

Operational Metrics

- PCAP parsing throughput
- feature extraction throughput
- supervised inference latency
- anomaly inference latency
- fusion latency
- end-to-end flow-to-alert latency
- p50 latency
- p95 latency
- p99 latency
- peak memory
- model size
- CPU usage

Threat Hunting Metrics

- alerts per 1,000 flows
- hypotheses generated
- alerts per hypothesis
- hypotheses containing multiple evidence sources
- hypothesis precision where ground truth is available
- case conversion rate
- false-positive reduction after correlation

Statistical Reliability

对主要指标计算：

- 95% bootstrap confidence interval
- 固定随机种子
- 至少 1,000 次 bootstrap

不得硬编码、伪造或预设最终结果。

12. Explainability

实现三层解释：

Global Explanation

- permutation importance
- tree-based importance
- feature distribution comparison

Local Detection Explanation

每条告警显示：

- top contributing features
- observed value
- expected/reference range
- effect direction
- reason codes

Reason codes 例如：

- ABNORMALLY_HIGH_PACKET_RATE
- LOW_IAT_VARIANCE
- PERIODIC_BEACONING_PATTERN
- HIGH_SYN_RATIO
- HIGH_RESET_RATIO
- STRONG_TRAFFIC_ASYMMETRY
- REPEATED_DESTINATION_ACTIVITY
- SHORT_CONNECTION_BURST
- SUPERVISED_HIGH_CONFIDENCE
- ANOMALY_HIGH_SCORE
- MULTIPLE_CORRELATED_ALERTS

Hypothesis Explanation

Hypothesis 必须明确区分：

- observed facts
- model inference
- assumptions
- alternative benign explanations
- recommended validation steps

不得把 feature importance 描述为因果关系。

13. Alert Correlation

实现基于时间窗口和实体的关联。

初始关联规则：

1. 同一 source IP 在短时间内触发多个扫描类告警。
2. 同一 source-destination pair 出现重复失败连接。
3. 同一 source 对多个 destination 产生异常连接。
4. 同一 destination 被多个 source 高频访问。
5. 周期性连接与异常流量同时出现。
6. 高风险监督预测与高 anomaly score 同时出现。
7. 多个低严重程度告警在同一实体上累计为高严重程度行为。

配置：

- correlation window
- minimum alerts
- entity keys
- decay factor
- score threshold

关联算法必须可测试、可配置和可解释。

14. Threat Hypothesis Generation

Hypothesis Engine 不使用自由生成文本作为核心判断。

采用：

- deterministic templates;
- structured evidence;
- correlation results;
- model explanations;
- optional MITRE mappings。

示例：

Title: Possible Network Reconnaissance Activity

Observed evidence:

- 46 short TCP connections
- 31 distinct destination ports
- high SYN ratio
- low completed handshake ratio
- events occurred within 18 seconds

Hypothesis: The source entity may be conducting service discovery or port reconnaissance.

Alternative explanations:

- vulnerability scanner
- monitoring system
- administrative inventory tool

Recommended validation:

- verify whether source belongs to authorized scanner
- inspect asset ownership
- compare activity with maintenance schedule
- review additional destination hosts
- check whether successful authentication followed the scan

不得将 hypothesis 自动标记为 confirmed。

15. Backend API

FastAPI 至少提供：

GET /health GET /system/status

POST /ingestion/pcap POST /ingestion/csv POST /ingestion/replay GET /ingestion/jobs GET /ingestion/jobs/{id}

GET /flows GET /flows/{id}

GET /alerts GET /alerts/{id} PATCH /alerts/{id}

GET /alert-groups GET /alert-groups/{id}

GET /hypotheses GET /hypotheses/{id} POST /hypotheses/{id}/create-case PATCH /hypotheses/{id}

GET /cases GET /cases/{id} POST /cases PATCH /cases/{id} POST /cases/{id}/notes POST /cases/{id}/feedback

GET /models GET /models/{id} POST /models/train POST /models/{id}/activate

GET /evaluation/latest GET /evaluation/{run_id}

API 必须：

- 使用 Pydantic validation；
- 有分页；
- 有错误处理；
- 有 OpenAPI 文档；
- 上传文件大小限制；
- 文件名和路径安全处理；
- 不允许任意路径读取；
- 不允许加载未经验证的 pickle；
- 对模型文件使用受控 registry。

16. Frontend Demonstration

使用 Streamlit 实现完整前端。

Page 1: Overview

显示:

- processed flows
- active alerts
- critical alerts
- open hypotheses
- open cases
- model versions
- p95 latency
- ingestion status
- recent activity timeline

Page 2: Data Ingestion

支持:

- 上传 PCAP;
- 上传 CSV;
- 选择 sample dataset;
- 启动 replay;
- 查看 ingestion progress;
- 查看错误;
- 查看 records processed。

Page 3: Traffic Explorer

显示:

- flow table;
- time range;
- protocol distribution;
- packet/byte distribution;
- source/destination relationships;
- benign/malicious/anomaly filters;
- flow detail。

Page 4: Alerts

显示:

- severity;
- detection engines;
- combined risk;
- reason codes;
- local explanation;
- involved entities;
- related alerts;
- analyst verdict。

Page 5: Threat Hunts

显示:

- generated hypotheses;
- confidence;
- evidence;
- assumptions;
- alternatives;
- MITRE mapping;
- recommended investigation;
- create case button。

Page 6: Cases

支持:

- case list;
- case details;
- investigation notes;
- evidence;
- status updates;
- analyst verdict;
- export case report。

Page 7: Model Lab

显示:

- active supervised model;
- active anomaly model;
- model versions;
- feature schema;
- train button;
- activate model;
- model comparison;
- global feature importance。

Page 8: Evaluation

显示:

- confusion matrix;
- ROC curve;
- PR curve;
- known vs unseen attack results;
- fusion comparison;
- latency distribution;
- model size;
- robustness results。

Page 9: System Health

显示：

- backend status;
- database status;
- worker status;
- CPU;
- memory;
- queue length;
- latest errors;
- model loading state。

前端必须能够在 sample data 模式下完整展示，不依赖实时抓包。

17. Security and Safety Boundaries

项目是防御性研究原型。

禁止实现：

- 未授权扫描;
- 漏洞利用;
- credential attacks against real systems;
- malware;
- persistence;
- destructive response;
- unrestricted flooding;
- automatic firewall modification;
- automatic account disabling;
- arbitrary command execution。

安全演示应采用：

- PCAP replay;
- synthetic flow generation;
- Docker-contained benign services;
- rate-limited behavioral simulation;
- dry-run response recommendations。

系统不得自动阻断网络。

可以提供 response recommendation，但默认只记录，不执行。

18. Reproducibility

所有实验必须记录：

- Git commit
- Python version
- dependency versions
- OS
- dataset checksum
- dataset version
- feature schema version
- split manifest
- model configuration
- random seeds
- threshold
- model artifact hash
- evaluation timestamp

Model bundle：

```
artifacts/models/<model_version>/ |—— supervised_model.joblib |—— anomaly_model.joblib
|—— preprocessing.joblib |—— feature_schema.json |—— thresholds.json |—— fusion_config.json
|—— training_config.yaml |—— metrics.json |—— model_card.md |—— dataset_manifest.json
|—— artifact_hashes.json
```

19. CLI Commands

实现统一 CLI：

aegishunt doctor aegishunt init-db

aegishunt ingest pcap <file> aegishunt ingest csv <file> aegishunt replay <file>

aegishunt dataset build aegishunt dataset validate aegishunt dataset report

aegishunt model train-supervised aegishunt model train-anomaly aegishunt model evaluate aegishunt
model list aegishunt model activate <version>

aegishunt hunt correlate aegishunt hunt generate-hypotheses

aegishunt api aegishunt frontend aegishunt demo

所有命令必须：

- 有 --help；
- 有参数校验；
- 有清晰日志；

- 有非零失败退出码；
 - 支持配置文件；
 - 不硬编码路径；
 - 不硬编码实验结果。
-

20. Test Requirements

Unit Tests

覆盖：

- configuration
- schemas
- packet parsing
- bidirectional flow key
- flow timeout
- feature calculation
- safe division
- dataset splitting
- leakage detection
- anomaly-score normalization
- score fusion
- severity mapping
- correlation
- hypothesis template selection
- case updates
- database repositories

Integration Tests

覆盖：

- PCAP → flows
- flows → feature dataset
- dataset → supervised model
- benign dataset → anomaly model
- models → detection
- detection → alert
- alerts → correlation group
- correlation group → hypothesis
- hypothesis → case
- database → API
- API → frontend data client

End-to-End Test

一条命令完成：

1. 初始化数据库；
2. 读取 sample PCAP；
3. 生成 flow；
4. 加载 sample/pretrained models；
5. 生成 detection results；
6. 创建 alerts；
7. 关联 alerts；
8. 生成 hypothesis；
9. 创建 case；
10. 验证 API 可返回结果。

要求：

- pytest 全部通过；
- 核心模块覆盖率目标 $\geq 80\%$ ；
- ruff 通过；
- mypy 通过；
- E2E 不依赖公网；
- E2E 不依赖 root；
- E2E 不执行真实攻击。

21. Final Deliverables

最终必须交付：

1. 完整源代码。
2. 完整 Git repository。
3. FastAPI backend。
4. Streamlit frontend。
5. SQLite sample database。
6. PCAP/CSV ingestion。
7. PCAP replay。
8. Flow feature extraction。
9. Supervised detection engine。
10. Unsupervised anomaly engine。
11. Fusion engine。
12. Alert correlation。
13. Threat hypothesis generation。
14. Investigation case management。
15. Analyst feedback。
16. Model registry。
17. Evaluation framework。
18. Sample data。
19. Pretrained demo models。
20. Automated tests。

21. Docker deployment。
 22. Setup documentation。
 23. Architecture documentation。
 24. Feature dictionary。
 25. Model card。
 26. Experiment protocol。
 27. Evaluation figures and tables。
 28. Demonstration guide。
 29. Known limitations。
 30. Thesis-ready technical materials。
-

22. Definition of Done

项目只有满足以下条件才算 100% 完成：

- 无需固定 target 即可运行；
 - sample PCAP 可以完整回放；
 - 可以从 PCAP 生成双向 flows；
 - 所有特征可复现；
 - 监督模型和异常模型可训练；
 - 两类模型可以同时推理；
 - fusion score 可配置；
 - 可以生成 security alerts；
 - 可以关联多个 alerts；
 - 可以生成结构化 threat hypothesis；
 - 可以创建和管理 case；
 - analyst feedback 可持久化；
 - 前端可演示完整流程；
 - API 有文档；
 - 测试通过；
 - 没有硬编码实验结果；
 - 没有数据泄漏；
 - 模型有版本管理；
 - 实验可以复现；
 - 项目可通过 Docker 或本地方式运行；
 - README 提供从安装到 Demo 的完整步骤；
 - 系统明确声明为研究原型，而非生产安全产品。
-

23. Development Rules

必须分阶段完成，不能一次性无检查地实现全部项目。

每个阶段开始时：

1. 阅读 docs/codex_progress.md。
2. 检查现有代码。
3. 说明本阶段目标。

4. 列出计划修改文件。
5. 不重复实现已有功能。

每个阶段结束时：

1. 运行 ruff。
2. 运行 mypy。
3. 运行相关 pytest。
4. 修复失败。
5. 更新 docs/codex_progress.md。
6. 总结修改文件。
7. 总结测试结果。
8. 说明已知限制。
9. 给出建议 commit message。
10. 停止，不自动进入下一阶段。

不得：

- 删除失败测试来让 CI 通过；
- 隐藏异常；
- 伪造命令执行结果；
- 伪造模型指标；
- 在测试集调参；
- 无说明地改变数据拆分；
- 将生成数据冒充真实网络数据；
- 将异常检测结果描述成确定攻击；
- 将 MITRE mapping 描述成已验证攻击归因。

现在仅执行 Phase 0。完成后停止。